

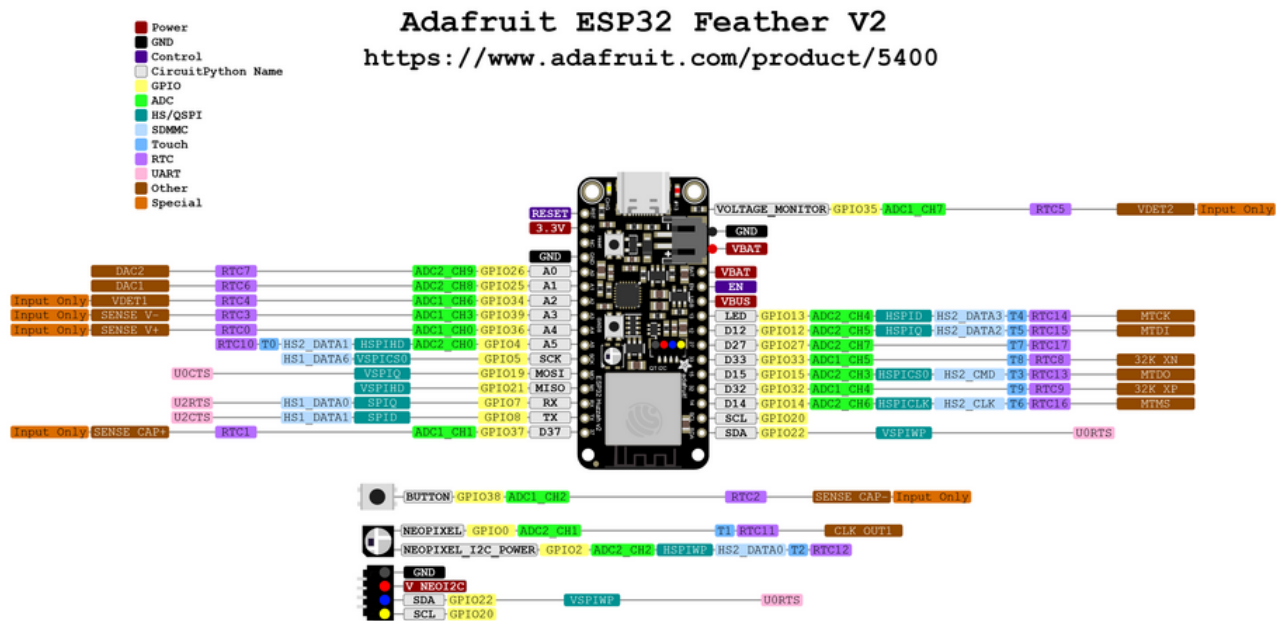


Figure 1: Adafruit ESP32 GPIO Pinout

Supplies

- Laptop with USB-C data cable, VS Code, and PlatformIO
- [Adafruit Feather ESP32 V2](#)
- Sears Lab: [HY3005F Benchtop DC power supply](#) + pair of spring-loaded test leads
- Sunfounder Universal Maker Kit [Tuned Torque \(TT\) Motor](#) + [L9110 Motor Driver](#)
- Sunfounder Universal Maker Kit [SG90 Servo Motor](#)

Part 1: ESP32 + Actuators

1. Set up (2 pt)

- Navigate to the Lab 4 folder in your ECSE 395 GitHub repository
- Create a file named `actuator_adventures.md` (1 pt)
- Edit the `actuator_adventures.md` in the Markdown editor of your choice, stating that this is your third assignment working with the ESP32 and that you will work on connecting actuators to ESP32. Also include all the steps you take to complete this lab assignment, indicate how you are uploading the code to the ESP32 (e.g., PlatformIO), and some additional details about your setup (e.g., Mac, Windows, Linux). Please also indicate what a reader will find in this

Feature	Servo Motor	DC Gear Motor (TT Motor)
Control Type	Uses a pulse-width modulation (PWM) signal to set and hold a specific position	Uses direct voltage and polarity to control speed and direction
Movement	Moves to a specific angle (e.g., 0-180° for standard servos) or rotates continuously in both directions (for continuous rotation servos)	Rotates continuously (not position-controlled)
Precision	High (can hold a specific angle)	Low (no feedback for position)
Torque	High for its size (due to internal gears)	Moderate, (gearbox increases torque)
Built-in components	Includes a motor, gear reduction system, and position feedback system (potentiometer and controller)	Motor with plastic or metal gears to reduce speed and increase torque
Feedback system	Yes (potentiometer in standard servos)	No (open-loop system)
Use cases	• Precise angular control (e.g., robotic arms, pan-tilt mechanisms) • Applications requiring specific positioning rather than continuous rotation	• Continuous rotation applications (e.g., wheeled robots, conveyor belts) • Simple motion tasks where precise position control is not required

Table 1: Servo vs. DC Gear Motor differences

repository (e.g., the code for actuator X can be found in XXX.cpp and the code for actuator Y can be found in YYY.cpp) (**1 pt**)

- In this lab, we'll be working with and learning about servo motors and DC gear motors (TT motors). These motors are similar, and both are used in robotics, but they have slight differences you should be aware of. Details about the differences between the two can be found in Table 1.

2. In-Class Task (**12 pts**)

- Prepare your laptop and ESP32.
- Find the *TT motor (DC Gear Motor)* in your Sunfounder Universal Maker kit.
- Following the instructions in the repository. Build your circuit with your ESP32, TT motor (DC Gear Motor), Motor Driver Module, breadboard, and jumper cables, wire according to the schematic (**NOTE:** you will need to put your motor driver module INTO the breadboard, DO NOT SOLDER).
- Set the benchtop DC Power Supply limited to 3V, 0.15A.
- Pay attention to the GPIO pinouts in Fig. 2 and **UNPLUG USB CABLE IF YOU SMELL BURNING/SMOKE!**
 - Please refer to the updated the schematic for the circuit connection.
- Take a picture of your circuit and include it in actuator_adventures.md. (**2 pts**)
- Upload your code “TT Motor.cpp” to your ESP32 microcontroller. Observe the behavior. Then change all of the following *one at a time*, to observe (and describe in your actuator_adventures.md how they affect the behavior) (**1 pt for changing the value, 1 pt for describing the affected behavior**):
 - Modify the analogWrite() to a different value (**2 pts**)

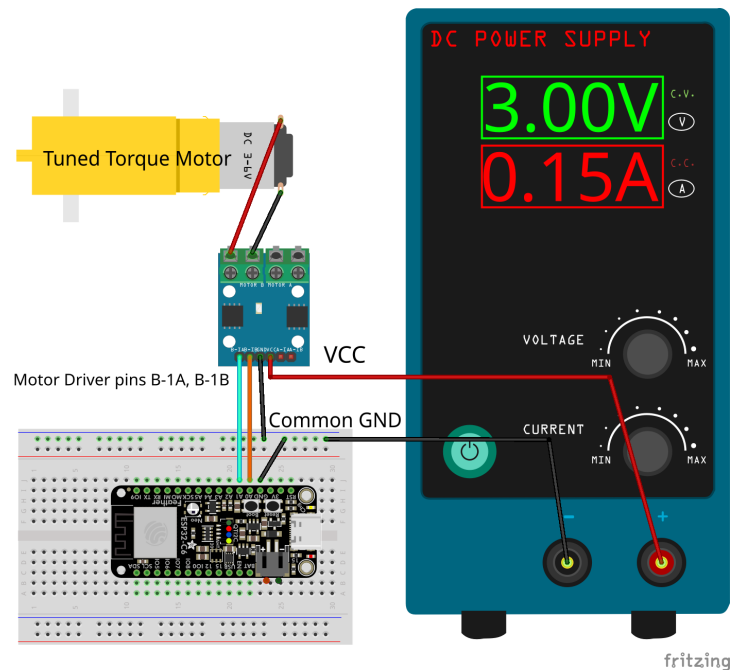


Figure 2: The wiring diagram of TT motor with ESP32

- Swap the `analogWrite()` values (2 pts)
- Modify the `delay()` (2 pts)
- **STOP:** if you were not able to get the accurate outcome from the previous step, please seek help from
 - Online Resources, classmates, instructors
- Follow the instructions and modify the skeleton code “TT Motor Rotate.cpp” so the motor runs clockwise for 5s, stops for 2s, runs counterclockwise for 5s and then stop for 2s, then loops this sequence. Comment what each line does along with your ABC123 CWRU ID. Save all your changes and push them to GitHub. (2 pts)
- Take a video of your motor running with the custom adjustments to prove it is working, and attach it to a comment in the Canvas assignment. (1 pt)
- Push your “TT Motor Rotate.cpp” file to your GitHub repo Lab 4 folder. Be sure to edit the `actuator_adventures.md` to include all relevant information, where to find the code, and how to implement it. (1 pt)
- **Extra credit:** Modify “TT Motor.cpp” to continuously *increase* and *decrease* motor speed in a loop. Save the code as “TT Motor EC.cpp” and push it to your GitHub repo’s Lab 4 folder. (discretionary points based on implementation)

3. Post-Class Task (16 pts)

- Find the *servo motor* in your Sunfounder Universal Maker Sensor kit
- Build your circuit with your ESP32, servo motor, breadboard, and jumper cables, wire according to the schematic in Fig. 3 (NOTE: you will need to connect your

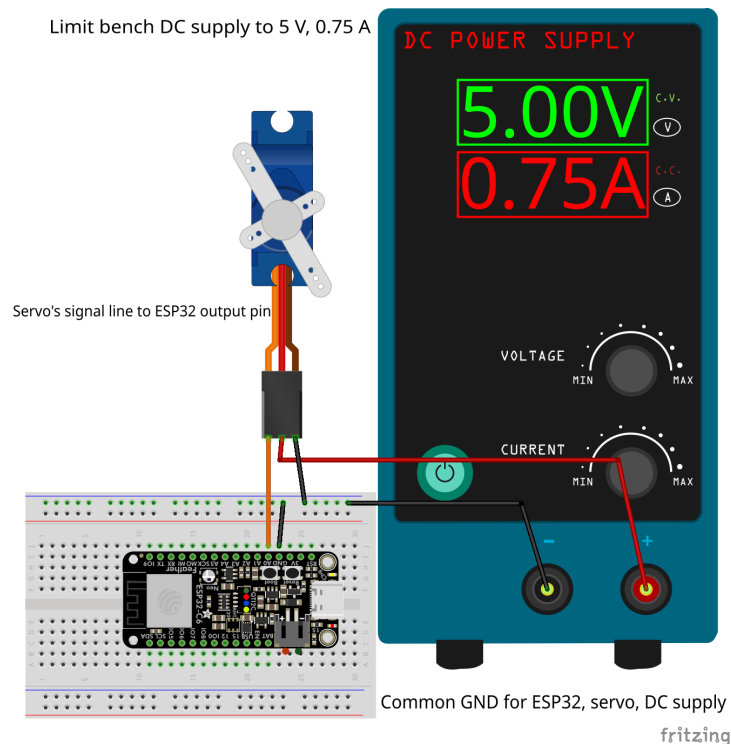


Figure 3: The wiring diagram of servo motor with ESP32

servo wires with **pin-pin jumper wires** and plug them INTO the breadboard, DO NOT SOLDER).

- Use the benchtop DC Power Supply limited to 5V, 0.75A.
- Pay attention to the GPIO pinouts in Fig. 3 and **UNPLUG USB CABLE IF YOU SMELL BURNING/SMOKE!**
- Take a picture of your circuit and include it in `actuator_adventures.md`. (2 pts)
- Upload the code “Servo Motor.cpp” to your ESP32 microcontroller. Observe the behavior. The servo should rotate from 0° to 180° and from 180° to 0°. Then change all of the following *one at a time* (and describe in your `actuator_adventures.md` how they affect the behavior) (1 pt for changing the value, 1 pt for describing the affected behavior):
 - `minPulseWidth` (2 pts)
 - `maxPulseWidth` (2 pts)
 - `setPeriodHertz` (2 pts)
 - Rotation range (2 pts)
 - `delay` (2 pts)
- **STOP:** if you were not able to get the accurate outcome from the previous step, please seek help from
 - Online Resources
 - Classmates
 - Instructors

- Instead of moving linearly, follow the instructions in the skeleton code “Servo Motor Random.cpp” and try to modify the code so the servo moves to random angles between 0° and 180° with different delays. (Don’t forget to add comments to explain the changes you are making!) Save the code and push your changes to GitHub. **(2 pts)**
- Take a video of your motor running with the custom adjustments to prove it is working. **(1 pt)**
- Push your code to the GitHub repo’s Lab 4 folder. Be sure to edit the `actuator_adventures.md` to include all relevant information, where to find the code, and how to implement it. **(1 pt)**
- **Extra credit:** Modify the code to gradually accelerate and decelerate the servo motion rather than moving at a constant speed. Save the code as “Servo Motor EC.cpp” and push it to GitHub. **(discretionary points based on implementation)**

Part 2: Time Reporting and Reflection (please add this to your `actuator_adventures.md` file) (5 pts)

1. How long did it take you to complete this assignment?
2. What level of difficulty would you associate with this assignment?
 - ☐ Low
 - ☐ Medium
 - ☐ High
3. If you associated medium/high difficulty with this assignment, what aspect did you find the most difficult?
4. How comfortable do you currently feel with the course content?
5. Do you have any additional information or feedback you would like to share with the instructors?

Submission Checklists

1. In the `actuator_adventures.md` file must including (but not limited to):
 - Your name
 - Assignment overview
 - What a reader will find in this repository (What is in the code file? Where can the reader find them?)

- Step to complete this lab (please add detail)
 - (a) Setup & Preparation
 - (b) In-Class & Post-Class Task
 - i. If you would like to do extra credit, please include
 - (c) Documentation
 - (d) etc.
 - How did you upload the code to ESP32 (please add detail)
 - (a) Tools Used
 - (b) Upload Process
 - (c) etc.
 - Pictures of your system setup
 - Parameter of TT motor and Servo motor, what did you observe by changing the parameters
 - Time reporting and reflection
2. All 4 code files, TT Motor.cpp, TT Motor Rotation.cpp, Servo Motor.cpp, Servo Motor Radom.cpp
- Comments all the changes you made to the code
 - Change the delay of your preference
 - Change all the parameters as instructed

If you haven't already, in your "Lab Notebook" repository, create a new Markdown file called "Week4.md." In the Week4.md file, include all updates of work you have done towards your semester project, including (but not limited to) your contributions to the needfinding presentation, the brainstorming milestone, and the preparations for your upcoming stakeholder meeting (where you will need to present your 3 fully-fleshed out concept ideas and get feedback on each one). Be especially careful to highlight your *individual contributions* vs. *what the group worked on as a whole* related to the project this week (clearly separate the two). **This is very important for the ABET Student Outcome related to teamwork.** Additional items to include would be any other elements of the project or project-related assignments you worked on. If nothing else is included, the assumption is that you did not contribute anything else to the semester project this week. As a polite reminder, your project grade will scale with your contributions. This will be verified through CATME peer evaluations and your weekly contributions in the lab notebook. Please update your weekly project log README.md to include your Week4.md file in a table of contents.